

UR5e ROS2 Verified Working Steps (ASCII, Consistent Format)

This document merges the verified working procedure and the Foolproof Startup Guide to control the real UR5e robot using ROS2 Humble and the ur_pick_and_place repository.

All commands are ASCII and safe to copy directly into the terminal.

Follow Steps from Foolproof Startup Guide

Execute the steps that correspond to launching the driver, powering the robot, releasing brakes, starting the External Control program, and activating controllers.

Launch the UR Driver

```
source /opt/ros/humble/setup.bash
source ~/workspace/ros_ur_driver/install/setup.bash
```

```
ros2 launch ur_robot_driver ur_control.launch.py \
  ur_type:=ur5e \
  robot_ip:=192.168.57.101 \
  launch_rviz:=false
```

Pendant: External Control (PC IP=192.168.57.1, Port=50002), Robot in Remote.

Power, Brakes, and Program

```
ros2 service call /dashboard_client/power_on std_srvs/srv/Trigger "{}"
ros2 service call /dashboard_client/brake_release std_srvs/srv/Trigger "{}"
ros2 service call /dashboard_client/play std_srvs/srv/Trigger "{}"
```

```
ros2 service call /io_and_status_controller/set_speed_slider \
  ur_msgs/srv/SetSpeedSliderFraction "{speed_slider_fraction: 0.5}"
```

Activate Controller

```
ros2 service call /controller_manager/switch_controller \
  controller_manager_msgs/srv/SwitchController "{ \
    start_controllers: ['scaled_joint_trajectory_controller'], \
    stop_controllers: ['joint_trajectory_controller', 'passthrough_trajectory_controller'], \
    strictness: 2, activate_asap: true, timeout: {sec: 5, nanosec: 0} }"
```

```
ros2 control list_controllers | grep scaled_joint_trajectory_controller
```

Launch MoveIt in Real Robot Mode

```
ros2 launch ur_moveit_config ur_moveit.launch.py \
  ur_type:=ur5e \
  launch_rviz:=true
```

Launch MoveIt connected to the live robot (no Gazebo).

Send Known-Good Goals

Option A - Safe Nudge (0.1 rad on wrist_3)

```
ros2 action send_goal /scaled_joint_trajectory_controller/follow_joint_trajectory \
  control_msgs/action/FollowJointTrajectory "{ \
    trajectory: { \
      joint_names: [ \
        'shoulder_pan_joint', 'shoulder_lift_joint', 'elbow_joint', \
        'wrist_1_joint', 'wrist_2_joint', 'wrist_3_joint' \
      ], \
      points: [{ \
        positions: [0.0, 0.0, 0.0, 0.0, 0.0, 0.10], \
        time_from_start: {sec: 2, nanosec: 0} \
      }] \
    } \
  }"
```

Option B - Ready Pose [0, -pi/2, +pi/2, 0, +pi/2, 0]

```
ros2 action send_goal /scaled_joint_trajectory_controller/follow_joint_trajectory \
  control_msgs/action/FollowJointTrajectory "{ \
    trajectory: { \
      joint_names: [ \
        'shoulder_pan_joint', 'shoulder_lift_joint', 'elbow_joint', \
        'wrist_1_joint', 'wrist_2_joint', 'wrist_3_joint' \
      ], \
      points: [{ \
        positions: [0.0, -1.5708, 1.5708, 0.0, 1.5708, 0.0], \
        time_from_start: {sec: 2, nanosec: 0} \
      }] \
    } \
  }"
```

Verification and Safety Checks

```
ros2 control list_controllers | grep scaled_joint_trajectory_controller
```

Ensure:

- # - External Control program running (Play symbol on pendant)
- # - Controller active
- # - Speed slider around 50%
- # - Emergency stop accessible

If robot does not move:

```
ros2 service call /dashboard_client/unlock_protective_stop std_srvs/srv/Trigger "{}"
ros2 service call /dashboard_client/play std_srvs/srv/Trigger "{}"
(re-run controller activation if necessary)
resend motion goal
```